ARTIFICIAL INTELLIGENCE · PRACTICAL SECTION 3

```
(deffacts …)
(defrule … ⇒ …)
(reset)
(run)
```

Introduction to CLIPS

Knowledge, facts and rules: building expert systems that reason

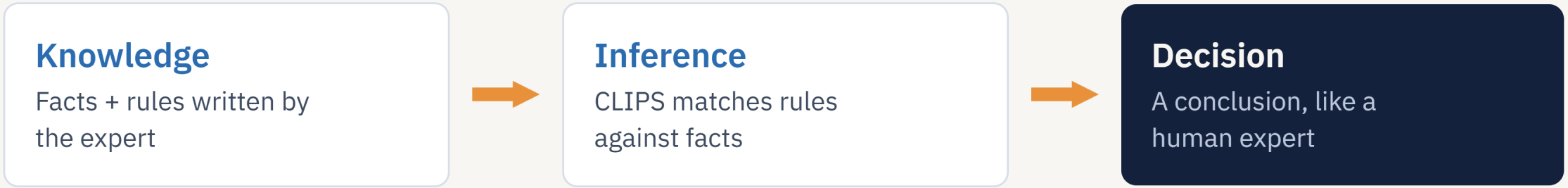
Under the supervision of **DR. Mohamed Kazim**

Prepared by **ENG. Moamen Yasser**

6.1

What is CLIPS?

CLIPS (C Language Integrated Production System) is a rule-based programming environment used to create **expert systems**. Knowledge is written as **facts** and **rules**, and the system makes decisions by logical inference.



Originally developed by **NASA,** widely used for knowledge representation, logical decision-making and automated reasoning.

6.2

Why CLIPS?

CLIPS is used when we need:

Knowledge-based reasoning

Reasoning with knowledge instead of numerical prediction.

Expert-like decisions

Systems that make decisions similar to human experts.

Clear separation

Knowledge (**facts**) is kept apart from logic (**rules**).

6.3

Key Features of CLIPS

Feature	Description
Rule-Based	Uses IF–THEN rules to represent knowledge
Forward & Backward Chaining	Infers conclusions from facts (forward) or works back from goals (backward)
Object-Oriented Support	Allows defining classes and instances
Knowledge Base Management	Stores and updates facts and rules dynamically
Cross-Platform & Extensible	Can be integrated with C/C++ or external systems
Standalone	Does not require external libraries

6.4

Components of CLIPS

Knowledge Base

Stores the domain facts and rules. Facts are static information; rules define how to reason about them.

Inference Engine

Matches rules against facts using forward chaining and fires the applicable rules to derive conclusions.

Working Memory

Holds the current state: the facts and intermediate results during reasoning.

Agenda

The list of rules ready to fire now. When several apply, the engine resolves the conflict with a strategy.

User Interface

A command-line prompt to interact with the system; can be connected to a GUI for end users.

In one line

Knowledge base + working memory feed the engine, the agenda says what fires next.

6.5

Functions, Actions and Commands

The three terms are used almost interchangeably, but each has a focus:

Function

Returns a value.

```
(+ 2 3) → 5
```

Action

No return value; does something as a side effect.

```
(printout t "Hi" crlf)
```

Command

Typed at the top-level prompt.

```
(reset) (run)
(set-strategy depth)
```

6.6

CLIPS Syntax Overview

Concept	Syntax example	Meaning
Fact	(student Ali)	A piece of information
Template fact	(deftemplate student (slot name) (slot grade))	Structured data
Rule	(defrule rule-name (condition) => (action))	Logic operation
Print output	(printout t "Hello" crlf)	Output text

Before we write facts and rules, let’s get comfortable with how CLIPS reads **any** expression.

BEFORE FACTS & RULES

CLIPS basics: using it as a calculator

Every CLIPS program is built from small expressions. Master them first, then rules become easy to read.

WHAT'S COMING

1. Prefix notation
2. Data types
3. Arithmetic & math functions
4. Comparison & logic
5. Variables & printing
6. If / else and loops
7. Strings & lists
8. Files: load, reset, run

The golden rule: prefix notation

```
(function arg1 arg2 …)
```

- Everything is inside **parentheses**
- The **operator comes first**, then the values
- Values are separated by **spaces**, not commas
- Comments start with **;**

In math	In CLIPS
5 + 3	(+ 5 3)
10 - 4	(- 10 4)
6 × 7	(* 6 7)
20 ÷ 4	(/ 20 4)
7 ÷ 2 → 3	(div 7 2)
7 % 2 → 1	(mod 7 2)
1+2+3+4	(+ 1 2 3 4)

Data types

Type	Example	Test function
Integer	42 -7 0	integerp
Float	3.14 2.5	floatp
Symbol	red Ali TRUE	symbolp
String	"Hello Ali"	stringp
Multifield	(create$ 1 2 3)	multifieldp

⚠ Test functions take **exactly one** argument. Build a list with **create$** first, so the whole list counts as one value.

```
CLIPS> (integerp 5)
TRUE
CLIPS> (numberp abc)
FALSE
CLIPS> (multifieldp
    (create$ 1 2 3))
TRUE
CLIPS> (multifieldp 1 2 3)
[ARGACCES1] expected
exactly 1 argument
```

A **symbol** is a single word with no quotes; a **string** is text in double quotes. Results **TRUE** and **FALSE** are symbols.

Arithmetic: numerical examples

```
(+ 5 3)
```
8

```
(- 10 4)
```
6

```
(* 6 7)
```
42

```
(+ 1 2 3 4)
```
10

```
(/ 20 4)
```
5.0

```
(/ 7 2)
```
3.5

```
(div 7 2)
```
3

```
(mod 7 2)
```
1

⚠ **/** always gives a float (5.0). Use **div** for integer division and **mod** for the remainder.

Nested expressions: inside first

Translate the math into CLIPS:

```
2 × (3 + 4) - 5
```



```
(- (* 2 (+ 3 4)) 5)
```

Each pair of parentheses is one operation.

How CLIPS evaluates it

```
(- (* 2 (+ 3 4)) 5)
(- (* 2 7) 5)
(- 14 5)
9
```

Built-in math functions

Expression	Result
`(sqrt 16)`	`4.0`
`(** 2 10)`	`1024.0`
`(abs -9)`	`9`
`(max 4 9 2)`	`9`
`(min 4 9 2)`	`2`

Expression	Result
`(round 3.6)`	`4`
`(integer 3.9)`	`3`
`(float 5)`	`5.0`
`(pi)`	`3.14159…`
`(* (pi) 2 2)`	`12.566…`

`**` is power, **integer** cuts the decimals, **round** rounds to the nearest. The last line is the area of a circle with radius 2.

Comparison and logic

Compare values

```
(> 5 3)          TRUE
(< 5 3)          FALSE
(= 4 4.0)        TRUE
(eq red red)     TRUE
(neq red blue)   TRUE
```

Numbers: **> < >= <= = <>** · Symbols and strings: **eq neq**

Combine conditions

```
(and (> 5 3) (< 2 1))
FALSE
(or (> 5 3) (< 2 1))
TRUE
(not (> 5 3))
FALSE
```

Same prefix rule: the operator comes first.

Variables with bind

```
(bind ?name value)
```

- A variable always starts with **?**
- **bind** stores a value in it
- Use it anywhere a value is expected

```
CLIPS> (bind ?x 10)
10
CLIPS> (* ?x ?x)
100
; Celsius to Fahrenheit
CLIPS> (bind ?c 25)
CLIPS> (+ (* ?c 1.8) 32)
77.0
```

Printing output with printout

```
(printout t "text" ?var (expression) crlf)
```

t

Send the output to the terminal (screen)

crlf

Go to a new line

Mix freely

Strings, variables and expressions

```
(printout t ?c " C = " (+ (* ?c 1.8) 32) " F" crlf)
25 C = 77.0 F
```

Decisions with if / then / else

```
(if condition
 then actions
 else actions)
```

```
(if (> 7 5) then big else small)
big
```

```
; student grade example
(bind ?g 75)
(if (≥ ?g 85)
  then (printout t "Excellent" crlf)
  else (if (≥ ?g 65)
         then (printout t "Good" crlf)
         else (printout t "Fail" crlf)))
```

```
Output: Good
```

Loops: while and loop-for-count

while

```
(bind ?i 1)
(while (<= ?i 5) do
  (printout t ?i " ")
  (bind ?i (+ ?i 1)))
```

Output: 1 2 3 4 5

loop-for-count

```
(loop-for-count (?i 1 5) do
  (printout t ?i " squared = "
    (* ?i ?i) crlf))
```

Output:
1 squared = 1
2 squared = 4
3 squared = 9 … 5 squared = 25

Strings and lists (multifields)

Strings	Result
`(str-cat "Hello " "Ali")`	`"Hello Ali"`
`(str-length "CLIPS")`	`5`
`(upcase "clips")`	`"CLIPS"`
`(sym-cat ab cd)`	`abcd`

Lists	Result
`(create$ 1 2 3)`	`(1 2 3)`
`(length$ (create$ a b c))`	`3`
`(nth$ 2 (create$ a b c))`	`b`
`(member$ b (create$ a b c))`	`2`

Functions that work on lists end with **$**. Positions start at **1**, not 0.

FROM CALCULATING TO REASONING

So far we told CLIPS what to compute. Now we give it knowledge and let it decide.

Facts

What we know: (weather hot)

Rules

What to do: IF hot THEN drink water

Inference

CLIPS matches and fires on (run)

6.7

Simple example: facts and a rule

Declaring facts

```
(deffacts initial-data
  (weather hot)
  (season summer)
)
```

deffacts lists the facts that enter working memory on **(reset)**.

Rule example

```
(defrule suggest-drink
  (weather hot)  ; IF
  =>             ; THEN
  (printout t
    "Recommendation: Drink Water!" crlf)
)
```

Left of **=>** = conditions; right of **=>** = actions.

6.7

Running the program

```
CLIPS> (reset)
CLIPS> (run)
```

OUTPUT

```
Recommendation: Drink Water!
```

1. **(reset)** puts (weather hot) and (season summer) in working memory
2. The engine sees that **suggest-drink** matches and puts it on the agenda
3. **(run)** fires it and the action prints the message

6.8

Structured facts using templates

```
(deftemplate student
  (slot name)
  (slot gpa)
)

(deffacts students
  (student (name "Ali") (gpa 3.5))
  (student (name "Sara") (gpa 2.1))
)
```

deftemplate

Defines the shape of a fact, like a record or a table row.

slot

A named field holding one value: name, gpa.

Why?

Order no longer matters and every value has a label.

6.9

Rule with slot matching

```
(defrule honor-student
  (student (name ?n)
           (gpa ?g&:(≥ ?g 3.0)))
  ⇒
  (printout t ?n " is an Honor Student!" crlf)
)

CLIPS> (reset)
CLIPS> (run)
```

```
OUTPUT Ali is an Honor Student!
```

`?n ?g`

Variables take the slot values of each matching fact

`&:`

“and the value must satisfy this test”

`(≥ ?g 3.0)`

The comparison from the basics: Ali (3.5) passes, Sara (2.1) does not

2.7 · DEFFUNCTION CONSTRUCT

Write your own reusable functions

7.1 Purpose: deffunction defines reusable functions that compute values or perform operations.

7.2 SYNTAX

```
(deffunction function-name
    (parameter-list)
  ; function body
)
```

Call it like any built-in: (function-name arg1 arg2)

7.3 · EXAMPLE 1

Add two numbers

```
(deffunction add-numbers (?a ?b)
  (printout t ?a " + " ?b " = " (+ ?a ?b) crlf)
)
```

CALL

```
(add-numbers 4 6)
```

OUTPUT

```
4 + 6 = 10
```

The body is one **printout** action mixing strings, the parameters and the expression **(+ ?a ?b)**.

7.3 · EXAMPLE 2

Check number type

```
(deffunction check-num (?n)
  (if (numberp ?n) then
      (if (= ?n 0) then Zero
        else (if (> ?n 0) then Positive
                  else Negative))
    else
      Non-numeric))
```

Call	Returns
(check-num 3)	Positive
(check-num -5)	Negative
(check-num 0)	Zero
(check-num one)	Non-numeric

numberp guards the comparisons, so a symbol like **one** never reaches **>** and causes no error.

7.3 · EXAMPLE 3

Recursive example: factorial

```
(deffunction factorial (?n)
  (if (= ?n 0) then 1
      else (* ?n (factorial (- ?n 1))))
)
```

```
(factorial 5) → 120
```

How it unfolds

```
5 × factorial(4)
5 × 4 × factorial(3)
5 × 4 × 3 × factorial(2)
5 × 4 × 3 × 2 × factorial(1)
5 × 4 × 3 × 2 × 1 × 1
= 120
```

The function calls itself with a smaller number until it reaches the **base case (= ?n 0)**.

SUMMARY

What you can do in CLIPS now

Basics

Prefix expressions, math, comparison, bind, printout, if, loops

Knowledge

deffacts, deftemplate and defrule; run with (reset) and (run)

Functions

deffunction for reusable logic, including recursion

Questions?

```
(printout t
  "Thank you!" crlf)
```

Thank You!

Any questions?

Under the supervision of **DR. Mohamed Kazim**

Prepared by **ENG. Moamen Yasser**